

CSC 212: Data Structures and Abstractions

02: Introduction to Analysis of Algorithms (part 1)

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Fall 2026



Analysis of algorithms

Problems, algorithms, programs

- Problem
 - ✓ a computational problem is a formalized task requiring a solution
 - ✓ well-defined input specification, expected output, formal constraints
- Algorithm
 - ✓ a precise, unambiguous sequence of computational steps
 - ✓ essential properties:
 - **correctness**: produces valid output for all valid inputs
 - **finiteness**: terminates after a finite number of steps
 - **determinism**: produces consistent results
 - **feasibility**: each step is executable
- Program implementation
 - ✓ concrete realization of an algorithm using a programming language

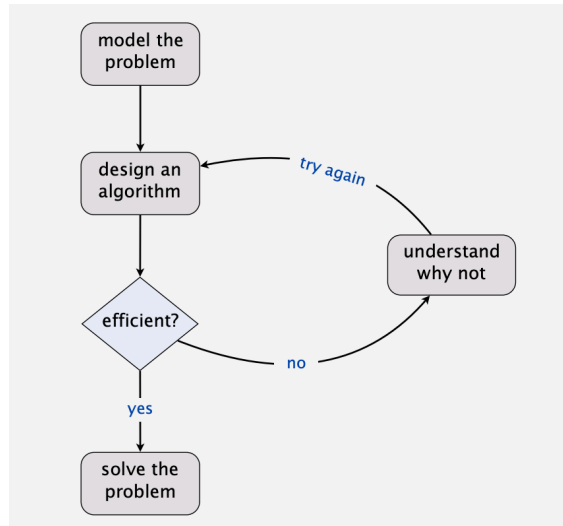
3

Q2-1

- A recipe step says: “Randomly decide whether to add salt.”
Which property of a well-defined algorithm does this violate?
- A. Correctness
 - B. Finiteness
 - C. Determinism
 - D. Feasibility
 - E. None of the above

4

Developing a usable algorithm



[COS 226 lectures, Princeton University]

5

Analysis of algorithms

• Definition

- ✓ the branch of CS that studies the **efficiency of algorithms** in terms of **time** and **space** complexity

• Objectives

- ✓ **performance prediction**: estimate running time growth with input size
- ✓ **resource optimization**: identify bottlenecks; minimize CPU time and memory
- ✓ **comparative evaluation**: provide standardized metrics to compare algorithms that solve the same problem

6

Why it matters?

• Scientific classification of algorithms

- ✓ groups algorithms by computational characteristics; complexity classes guide design and approximation strategies

• Performance prediction and resource utilization

- ✓ anticipates behavior before implementation, avoiding costly rewrites

• Time and space complexity

- ✓ critical where processing power or memory is constrained

• Theoretical guarantees

- ✓ provides upper-bounds for reliable, predictable system design

7

Two approaches

• Empirical analysis (today)

- ✓ implement the program
- ✓ systematic testing across varied input sizes
- ✓ statistical analysis of the measurements => **hypothesis**
- ✓ validation through **prediction**

• Theoretical analysis (next lecture)

- ✓ **mathematical modeling** of time and space complexity
- ✓ no implementation, no execution

These are not rivals. They are the two halves of the scientific method applied to code.

8

Q2-2

• You want to compare two sorting algorithms without writing or running any code. Which approach are you using?

- A. Empirical analysis
- B. Theoretical analysis
- C. Both, simultaneously
- D. Neither, this isn't possible
- E. None of the above

9

Empirical analysis

Example: evaluating a polynomial

• A polynomial of degree n is written:

$$p(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n = \sum_{i=0}^n a_ix^i$$

• **The task:** given the coefficient array $a[0..n]$ and a value x , compute $p(x)$

Two programs follow. Both take the same array of coefficients and the same x . Both return the same number. One of them does dramatically more work than the other, can you tell which by glancing at them?

11

Solution 1 (rebuild each power)

```
double poly1(const double a[], int n, double x) {
    double p = 0.0;
    for (int i = 0 ; i <= n ; i++) {
        double xp = 1.0;
        for (int j = 1 ; j <= i ; j++) {
            xp *= x;
        }
        p += a[i] * xp;
    }
    return p;
}
```

Check the redundant computation with $n=4$. The inner loop rebuilds x^i from scratch every time, having just computed x^{i-1} one iteration earlier and thrown it away.

12

Trace poly1 by hand ($n = 4, x = 2$)

- Ignore p and the coefficients. Watch one variable: xp

outer i	inner loop runs	xp goes	mults	repeated?
0	never	1	0	—
1	j = 1	1 $\Rightarrow$ 2	1	1 new
2	j = 1, 2	1 $\Rightarrow$ 2 $\Rightarrow$ 4	2	1 new, 1 redone
3	j = 1, 2, 3	1 $\Rightarrow$ 2 $\Rightarrow$ 4 $\Rightarrow$ 8	3	1 new, 2 redone
4	j = 1, 2, 3, 4	1 $\Rightarrow$ 2 $\Rightarrow$ 4 $\Rightarrow$ 8 $\Rightarrow$ 16	4	1 new, 3 redone

Total: 10 multiplications to produce 4 new values. Six of the ten recompute a number the program held one iteration earlier and threw away.

13

Solution 2 (carry the power forward)

```
double poly2(const double a[], int n, double x) {
    double p = a[0];
    double xp = 1.0;
    for (int i = 1; i <= n; i++) {
        xp *= x;
        p += a[i] * xp;
    }
    return p;
}
```

Insight: $x^i = x \cdot x^{i-1}$, so carry the power forward instead of rebuilding it

Same output to every digit. Same loop structure at a glance. A fundamentally different amount of work.

14

Q2-3

- At $n = 1,000$, how many times more multiplications does **poly1** do?

poly1 performs $\frac{n(n+1)}{2}$ multiplications

poly2 performs n

- About $2x$
- About $500x$
- About $1000x$
- About $1,000,000x$
- None of the above

15

Timing

```
#include <iostream>
#include <iomanip>
#include <chrono>
#include <cstdlib>
#include <vector>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        std::cerr << "Usage: " << argv[0] << " <n> <alg>\n";
        return 1;
    }

    int n = std::stoi(argv[1]);
    int alg = std::stoi(argv[2]);
    const double x = 0.999999;

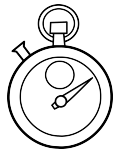
    std::vector<double> a(n + 1); // random coefficients in [-1, 1]
    std::srand(42); // fixed seed = reproducible
    for (int i = 0; i <= n; i++)
        a[i] = 2.0 * ((double) std::rand() / RAND_MAX) - 1.0;

    double p;
    auto start = std::chrono::high_resolution_clock::now();
    if (alg == 1) p = poly1(a.data(), n, x);
    if (alg == 2) p = poly2(a.data(), n, x);
    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double> elapsed = end - start;
    std::cout << std::fixed << std::setprecision(10);
    std::cout << p << " " << (double) elapsed.count() << std::endl;

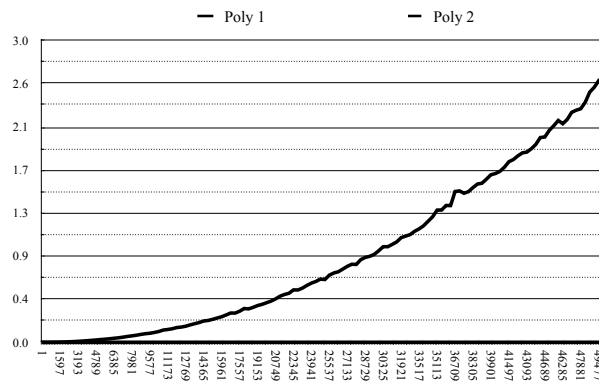
    return 0;
}
```

\$ g++ -std=c++11 -O0 poly.cpp -o prog



16

Hypothesis, Prediction, Validation



The scientific method, applied to code:

- **Observe:** run the programs across a range of input sizes
- **Hypothesize:** propose a model, e.g. $T(n) = cn^2$
- **Predict:** use the model to forecast a running time you have not yet measured
- **Validate:** measure it. If the prediction holds, the model survives. If not, revise.

A hypothesis earns its keep by predicting a measurement you haven't taken yet.

```
$ seq 1 399 50000 | while read n; do printf "%s\t" "$n"; ./prog $n 1; done > p1.txt
$ seq 1 399 50000 | while read n; do printf "%s\t" "$n"; ./prog $n 2; done > p2.txt
```

17

Q2-4

- Consider the same timing program, change one flag and run it again (-O0 => -O3)
 - ✓ not one line of your code changes, same algorithm, same coefficients, the only difference is what the compiler generates
- Fill this table from your own machine:

n	poly1 -O0	poly1 -O3	speedup T_O0 / T_O3
131,072			
262,411			
524,288			

How much faster is -O3?

18

Q2-5

- A student asks an AI assistant to “benchmark these two functions.” It produces this code and reports that `poly2` is about 8x faster? What is the most serious flaw in this benchmark?

```
int main() {
    double a[101];
    for (int i = 0; i <= 100; i++) a[i] = 1.0;

    auto t1 = std::chrono::high_resolution_clock::now();
    poly1(a, 100, 0.5);
    auto t2 = std::chrono::high_resolution_clock::now();
    poly2(a, 100, 0.5);
    auto t3 = std::chrono::high_resolution_clock::now();

    std::cout << "poly1: " << (t2-t1).count() << "\n";
    std::cout << "poly2: " << (t3-t2).count() << "\n";
}
```

- It measures at a single input size, so it cannot reveal how either function grows
- The return values are unused, so -O2 may delete both calls entirely
- A single untimed run at small n is dominated by noise and cold-cache effects
- All of the above are genuine flaws
- None of the above, the benchmark is sound

19

Limitations of empirical analysis

- Implementation challenges
 - ✓ you must implement every algorithm you want to compare, and implementation quality varies
- Experimentation challenges
 - ✓ designing representative test cases; long execution cycles; sensitivity to input distribution
- Environmental dependencies
 - ✓ compiler flags (-O0, -O1, -O2, -O3), HW differences (AVX, branch prediction, cache sizes), OS scheduling, background load
- Reproducibility
 - ✓ timings fluctuate; exact conditions are hard to replicate; results may not generalize beyond the range tested

20